MAT700 Lecture3 – Example of Data Mining

A. Balinsky

Cardiff School of Mathematics

Example of Data Mining: Finding Similar Items

A fundamental data-mining problem is to examine data for "**similar**" items.

For example, we would like to look at a collection of Web pages and to find near-duplicate pages.

These pages could be plagiarisms, for example, or they could be mirrors that have almost the same content but differ in information about the host and about other mirrors.

- We begin by phrasing the problem of similarity as one of finding sets with a relatively large intersection.
- We show how the problem of finding textually similar documents can be turned into such a set problem by the technique known as "*shingling*."
- Then, we introduce a technique called "*minhashing*," which compresses large sets in such a way that we can still deduce the similarity of the underlying sets from their compressed versions.

Jaccard Similarity of Sets

We shall focus initially on a particular notion of "similarity": **the similarity of sets** by looking at the relative size of their intersection. This notion of similarity is called "Jaccard similarity".

We then examine some of the uses of finding similar sets. These include finding textually similar documents and collaborative filtering by finding similar customers and similar products.

Definition (Jaccard Similarity)

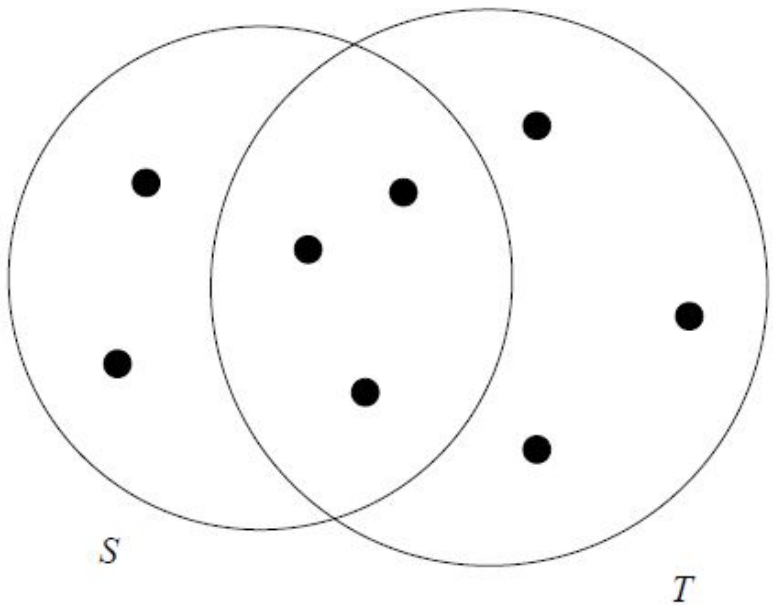
The *Jaccard similarity* of sets S and T is

$$|S \cap T|/|S \cup T|,$$

that is, the ratio of the size of the intersection of S and T to the size of their union.
We shall denote the Jaccard similarity of S and T by $SIM(S, T)$.

Example

In Fig. below we see two sets S and T . There are three elements in their intersection and a total of eight elements that appear in S or T or both. Thus, $SIM(S, T) = 3/8$.



Similarity of Documents

- An important class of problems that Jaccard similarity addresses well is that of finding *textually similar documents* in a *large corpus* such as the Web or a collection of news articles.

- We should understand that the aspect of similarity we are looking at here is **character-level** similarity, not "similar meaning," which requires us to examine the words in the documents and their uses.

- Textual similarity has important uses.

  Many of these involve finding *duplicates* or *near duplicates*.

  First, let us observe that testing whether two documents are exact duplicates is easy; just compare the two documents character-by-character, and if they ever differ then they are not the same.

  However, in many applications, the documents are not identical, yet they share large portions of their text. Here are some examples:

Similarity of Documents. Examples

- **Plagiarism**: Finding plagiarized documents tests our ability to find textual similarity. The plagiarizer may extract only some parts of a document for his own. He may alter a few words and may alter the order in which sentences of the original appear. Yet the resulting document may still contain 50% or more of the original. No simple process of comparing documents character by character will detect a sophisticated plagiarism.
- **Mirror Pages**: It is common for important or popular Web sites to be duplicated at a number of hosts, in order to share the load. The pages of these mirror sites will be quite similar, but are rarely identical.
  It is important to be able to detect similar pages of these kinds, because search engines produce better results if they avoid showing two pages that are nearly identical within the first page of results.

- **Articles from the Same Source:** It is common for one reporter to write a news article that gets distributed, say through the Associated Press, to many newspapers, which then publish the article on their Web sites.
  News aggregators, such as Google News, try to find all versions of such an article, in order to show only one, and that task requires finding when two Web pages are textually similar, although not identical.

Collaborative Filtering

Another class of applications where *similarity of sets* is very important is called collaborative filtering, a process whereby we *recommend to users items that were liked by other users who have exhibited similar tastes*.

Examples:

- **On-Line Purchases**: **Amazon.com** has millions of customers and sells millions of items.
  We can say two customers are *similar* if their sets of purchased items have a high Jaccard similarity.

  Likewise, two items that have sets of purchasers with high Jaccard similarity will be deemed similar.

  Jaccard similarities need not be very high to be significant.

- **On-Line Purchases**:

  Collaborative filtering requires several tools, in addition to finding similar customers or items:

  For example, two Amazon customers who like science-fiction might each buy many science-fiction books, but only a few of these will be in common.

  However, by combining similarity finding with **clustering** , we might be able to discover that science fiction books are mutually similar and put them in one group.

  Then, we can get a **more powerful notion of customer-similarity** by asking whether they made purchases within many of the same groups.

- **Movie Ratings**: NetFlix records which movies each of its customers rented, and also the ratings assigned to those movies by the customers.

  We can see movies as similar if they were rented or rated highly by many of the same customers, and see customers as similar if they rented or rated highly many of the same movies.

  The same observations that we made for Amazon above apply in this situation: similarities need not be high to be significant, and clustering movies by genre will make things easier.

- **Jaccard similarity for bags**: the matter of ratings introduces a new element.
  If ratings are 1-to-5-stars, put a movie in a customer's set n times if they rated the movie n-stars. Then, use *Jaccard similarity for bags* when measuring the similarity of customers.

Definition (The Jaccard Similarity for Bags)

The Jaccard similarity for bags B and C is defined by counting an element n times in the intersection if n is the minimum of the number of times the element appears in B and C. In the union, we count the element the *sum* of the number of times it appears in B and in C.

Example

The bag-similarity of bags $\{a, a, a, b\}$ and $\{a, a, b, b, c\}$ is 1/3. The intersection counts a twice and b once, so its size is 3. The size of the union of two bags is *always the sum* of the sizes of the two bags, or 9 in this case.

Shingling of Documents

The most effective way to represent documents as sets, for the purpose of identifying lexically similar documents is to construct from the document the set of short strings that appear within it.

If we do so, then documents that share pieces as short as sentences or even phrases will have many common elements in their sets, **even if those sentences appear in different orders** in the two documents.

Definition (*k*-Shingles)

A document is a string of characters. Define a *k-shingle* for a document to be any substring of length *k* found within the document. Then, we may associate with each document the set of k-shingles that appear one or more times within that document.

Example

Suppose our document D is the string ***abcdabd***, and we pick $k = 2$. Then the set of 2-shingles for D is $\{ab, bc, cd, da, bd\}$.

Note that the substring ***ab*** appears twice within D, but appears only once as a shingle.

A variation of shingling produces a bag, rather than a set, so each shingle would appear in the result as many times as it appears in the document. However, we shall not use bags of shingles here.

There are several options regarding how *white space* (blank, tab, newline, etc.) is treated. It probably makes sense to replace any sequence of one or more white-space characters by a single blank.

Choosing the Shingle Size

We can pick k to be any constant we like. However, if we pick k too small, then we would expect most sequences of k characters to appear in most documents.

How large k should be depends on how long typical documents are and how large the set of typical characters is.

The important thing to remember is:

k should be picked large enough that the probability of any given shingle appearing in any given document is low.

If our corpus of documents is emails, picking $k = 5$ should be fine:

Suppose that only letters and a general white-space character appear in emails.

Then there would be $27^5 = 14,348,907$ possible shingles. Since the typical email is much smaller than 14 million characters long, we would expect $k = 5$ to work well, and indeed it does.

However, the calculation above is a bit more subtle.

Surely, more than 27 characters appear in emails.

However, all characters do not appear with equal probability.

Common letters and blanks dominate, while "z" and some other letters are rare.

Thus, even short emails will have many 5-shingles consisting of common letters, and the chances of unrelated emails sharing these common shingles is greater than would be implied by the calculation in the slide above.

A good rule of thumb is to imagine that there are only 20 characters and estimate the number of k-shingles as 20^k. For large documents, such as research articles, choice $k = 9$ is considered safe.

Similarity-Preserving Summaries of Sets

Sets of shingles are **large**.
If we have millions of documents, it may well not be possible to store all the shingle-sets in main memory.

There is another serious concern: even if the sets fit in main memory, the number of pairs may be too great for us to evaluate the similarity of each pair.

Our goal in this section is to replace large sets by much smaller representations called "signatures".

The **important property** we need for signatures is that we can compare the signatures of two sets and estimate the Jaccard similarity of the underlying sets from the signatures alone.

Matrix Representation of Sets

Before explaining how it is possible to construct small signatures from large sets, it is helpful to visualize a collection of sets as their ***characteristic matrix***:

The columns of the matrix correspond to the sets, and *the rows correspond to elements of the universal set* from which elements of the sets are drawn.

There is a **1** in row r and column c if the element for row r is a member of the set for column c. Otherwise the value in position (r, c) is **0**.

Example

Fig. below is an example of a matrix representing sets chosen from the universal set $\{a, b, c, d, e\}$.
Here, $S_1 = \{a, d\}$, $S_2 = \{c\}$, $S_3 = \{b, d, e\}$, and $S_4 = \{a, c, d\}$.
The top row and leftmost columns are not part of the matrix, but are present only to remind us what the rows and columns represent.

Element	S_1	S_2	S_3	S_4
a	1	0	0	1
b	0	0	1	0
c	0	1	0	1
d	1	0	1	1
e	0	0	1	0

Minhashing

The **signatures** we desire to construct for sets are composed of the results of a large number of calculations, say several hundred, each of which is a "**minhash**" of the characteristic matrix.

Definition (Minhash)

To **minhash** a set represented by a column of the characteristic matrix, pick a permutation of the rows. The minhash value of any column is the number of the first row, in the permuted order, in which the column has a **1**.

Example

Let us suppose we pick the order of rows *beadc* for the matrix above. This permutation defines a minhash function *h* that maps sets to rows.

Element	S_1	S_2	S_3	S_4
b	0	0	1	0
e	0	0	1	0
a	1	0	0	1
d	1	0	1	1
c	0	1	0	1

$$h(S_1) = a, \quad h(S_2) = c, \quad h(S_3) = b, \quad h(S_4) = a.$$

There is a remarkable *connection* between *minhashing* and *Jaccard similarity* of the sets that are minhashed.

Minhashing and Jaccard Similarity

Theorem

The probability that the minhash function for a random permutation of rows produces the same value for two sets equals the Jaccard similarity of those sets.

Proof.

If we restrict ourselves to the columns for sets S_1 and S_2, then rows can be divided into three classes:

1. Type X rows have 1 in both columns.
2. Type Y rows have 1 in one of the columns and 0 in the other.
3. Type Z rows have 0 in both columns.

$$SIM(S_1, S_2) = |X|/(|X| + |Y|).$$

$$p(h(S_1) = h(S_2)) = |X|/(|X| + |Y|).$$



Minhash Signatures

- Think of a collection of sets represented by their characteristic matrix M.
- To represent sets, we pick at random some number n of permutations of the rows of M.
  Perhaps 100 permutations or several hundred permutations will do.
- Call the **minhash** functions determined by these permutations $h_1, h_2, \ldots, h_n$.

- From the column representing set S, construct the minhash signature for S, the **vector** $[h_1(S), h_2(S), \ldots, h_n(S)]$. We normally represent this list of hash-values as a column.

- Thus, we can form from matrix M a **signature matrix**, in which the ith column of M is replaced by the minhash signature for (the set of) the ith column.
  Note that the signature matrix has the same number of columns as M but only n rows.